

4. CNN

Q.1 Explain padding and stride.

⇒

1) Padding

⇒

- Padding is a technique used to preserve the spatial dimension of the input image after convolution operations on a feature map.
- Padding involves adding extra pixels around the border of the input feature map before convolution.
- This can be done in two ways -

a) Valid Padding : In valid padding, no padding is added to the input feature map, and the output feature map is smaller than the input feature map.

b) Same Padding :

⇒ In the same padding, padding is added to the input feature map such that the size of the output feature map is same as the input feature map.

- The number of pixels to be added for padding can be calculated based on the size of the kernel and the desired output of the feature map size.
- The most common padding value is zero-padding, which involves adding zeros to the borders of input feature map.

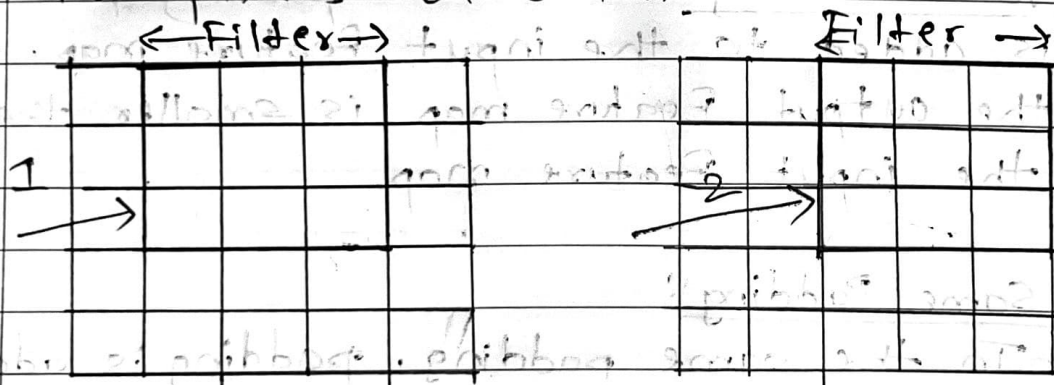
2) Stride

⇒ The number of rows and columns traversed per slide are referred as stride.

- It can be thought as by how many pixels we want our filter to move as it slides across the image.

→ Stride of 1, both for height and width can be used or sometimes larger strides are used.

- When the stride is one the filter is moved at one pixel at a time and when the stride is two the filter is moved to two pixels at a time.



→ Stride in CNN technique which has two main features.

→ The first is to reduce the size of the output feature map.

→ Formula:

$$\text{Output size} = \text{Input size} - \text{Kernel size} + 2P + 1$$

Stride

Q.2 Explain convolution and pooling operation with an example.

⇒

• Convolution operation

- Convolution is fundamental operation in CNNs, used to extract features from input images or data.
- It involves sliding a small filter over the input image and compute element-wise multiplication followed by summation.
- This operation helps detect edges, textures and patterns in the image.

a	b	c	d
e	f	g	h
i	j	k	l

w	x
y	z

Input Matrix

Formula :-

$$S_{ij} = (I * K)_{ij}$$

$$= \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i+a, j+b} \cdot K_{a+b}$$

m, n = rows and columns.

- For e.g.,

Let the input image be 3×3 matrix:

$$\text{Image} = \begin{bmatrix} 1 & 2 & 0 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$\text{Filter} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- Convolution process begins by placing the filter at the top-left corner of the input.

- For the first position,

$$\begin{aligned} & (1 \times 1) + (0 \times 2) + (4 \times 0) + (5 \times 1) \\ &= 1 + 0 + 0 + 5 \\ &= 6 \end{aligned}$$

- For the second position,

$$\begin{aligned} & (2 \times 1) + (0 \times 0) + (5 \times 0) + (1 \times 6) \\ &= 2 + 0 + 0 + 6 \\ &= 8 \end{aligned}$$

- For the third position,

$$\begin{aligned} & (4 \times 1) + (5 \times 0) + (7 \times 0) + (8 \times 1) \\ &= 4 + 0 + 0 + 8 \\ &= 12 \end{aligned}$$

- For the fourth position,

$$(5 \times 1) + (6 \times 0) + (8 \times 0) + (9 \times 1)$$

$$= 5 + 0 + 0 + 9$$

$$= 14$$

- Final Result :-

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

• Pooling Operation

- Pooling is a down sampling operation that reduces the spatial feature dimensions of the feature map while retaining the most important information.
- It helps make CNNs translation invariant, reducing sensitivity to small variation in input image.

- Types :-

- a) Max-pooling
- b) Average pooling

a) Max-pooling

⇒ Max pooling selects the maximum element from the region of the feature map covered by the filter.

- For e.g.,

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 8 & 7 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 16 & 15 \end{bmatrix} \xrightarrow[\text{Filter} = (2 \times 2)]{\text{Max pool, stride} = (2 \times 2)} \begin{bmatrix} 6 & 8 \\ 14 & 16 \end{bmatrix}$$

b) Average Pooling

$\Rightarrow$ Average pooling computes the average of the elements present in the region of feature map covered by the filter.

- For e.g.,

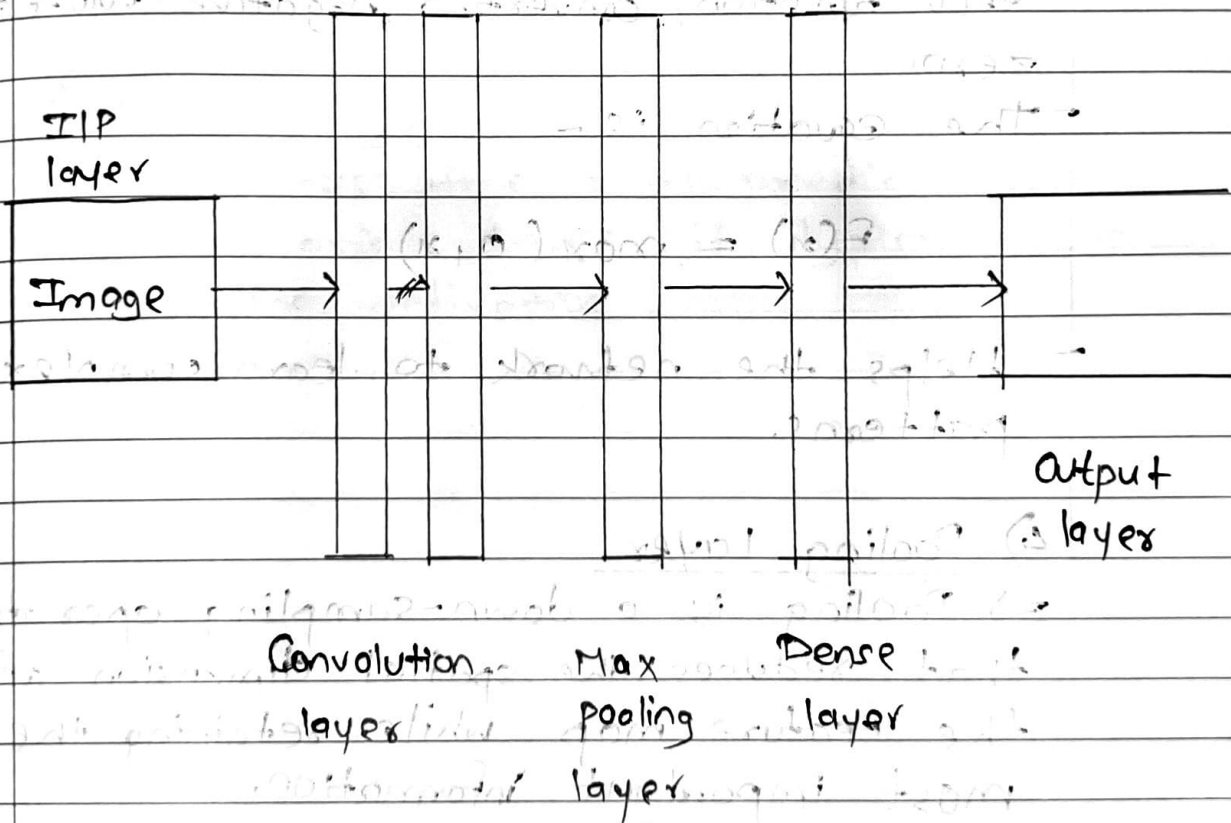
$$\begin{bmatrix} 2 & 2 & 7 & 3 \\ 9 & 4 & 6 & 1 \\ 8 & 5 & 2 & 4 \\ 3 & 1 & 2 & 6 \end{bmatrix} \xrightarrow{\text{Avg. Pool}} \begin{bmatrix} 4.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}$$

- Why are Pooling layers important?

- Dimensionality Reduction: Pooling layer reduces the spatial size of the feature maps, which decreases the no. of parameters & computations.
- Translation invariance: Pooling helps the network become invariant to small translations or distortions in input image.
- Overfitting prevention: By reducing spatial dimension, pooling layers help prevent overfitting by providing a form of regularization.
- Feature hierarchy: Pooling layer helps build a hierarchical representation of features, where lower layer capture fine details & higher layer capture more abstract and global features.

Q.9 Explain the architecture of the CNN with the help of diagram.

- ⇒
- A CNN is a deep learning model designed for image recognition and processing.
 - CNN consists of multiple layers like the input layer, convolution layer, pooling layer and fully connected layer.
 - CNNs are widely used in computer vision applications due to their effectiveness in processing visual data.



1) Input layer

- ⇒ Accepts the raw image data, typically in the form of $\{h \times w \times c\}$
- e.g.: $\{64 \times 64 \times 3\}$
- Normalizes the pixel values (0 to 255) to a range like $[0, 1]$ for better training efficiency.
or $[-1, +1]$

2) Convolution layer

⇒ The core building block of CNNs, where kernels slide over the input image.

- Computes the dot product between the filter and image pixels to generate feature maps.
- Helps detect edges, textures, shapes and other important spatial features.

3) Activation layer (ReLU)

⇒ Introduces non-linearity by applying the ReLU function, converting negative values to zero.

- The equation is -

$$F(x) = \max(0, x)$$

- Helps the network to learn complex patterns.

4) Pooling Layer

⇒ Pooling is a down-sampling operation that reduces the spatial dimension of the feature map while retaining the most important information.

- These are two types -

a) Max pooling: Max pooling selects the maximum element from the region of the feature map covered by the filter.

b) Average Pooling : Average pooling computes the average of the elements present in the region of feature map covered by the filter.

5) Fully Connected Layer

- ⇒ The output of convolutional and pooling layer is flattened into 1D vector.
- Each neuron in this layer is connected to all neurons in the previous layer.
- Outputs final layer predictions using softmax.

6) Output layer

- ⇒ Produces the final predictions for classification tasks, it typically uses softmax activation.
- The output from the fully connected layers is then fed into a logistic function or classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

- Advantages of CNN

- 1) Good at detecting patterns and features in images, videos or audios.
- 2) Robust to translation, rotation and scaling invariance.
- 3) End-to-end training, no need for manual feature extraction.

4) Can handle large amounts of data and achieve high accuracy.

- Disadvantages of CNN:

1) Computationally expensive to train and require a lot of memory.

2) Can be prone to overfitting if not enough data or proper regularization is used.

3) Requires large amount of labeled data.

Q.4 Explain CNN architecture in detail. Consider we have input volume of $32 \times 32 \times 3$ for a layer in CNN and there are ten 5×5 filter with stride 1 and pad 2; calculate the no. of parameters in this layer of CNN.

⇒ Solⁿ:-

$$\text{Input size} = 32 \times 32 \times 3$$

$$\text{Filter} = 5 \times 5$$

$$\text{No. of Filters} = 10$$

$$\text{Padding} = 2$$

$$\text{Stride} = 1$$

$$\therefore \text{Output size} = N - F + 2P + 1$$

$$H_{out} = 32 - 5 + (2 \times 2) + 1$$

$$\therefore H_{out} = 32$$

$$\therefore W_{out} = 28 - 5 + (2 \times 2) + 1$$

$$= 32$$

- Since there are 10 filters, the depth of output volume is 10.
- Thus, output volume is -
 $32 \times 32 \times 10$

2) Parameters per Filter:

Each filter has $5 \times 5 \times 3$ dimension

The number of parameters per filter is $\Rightarrow 5 \times 5 \times 3 = 75$

Each filter has 1 bias term.

Total parameter per filter $= 75 + 1$

$$= 76$$

3) Total parameter for 10 filter:

$$76 \times 10$$

$$= 760 \text{ parameters.}$$

Q.5 (Dec 24) Q4. b. Given input feature map of size $64 \times 64 \times 32$.
 $\Rightarrow$ Soln:-

$$\text{Input} = 64 \times 64 \times 32$$

$$\text{Input channels} = 32$$

$$\text{Input spatial size} = 64 \times 64$$

$$\text{No. of filters} = 64$$

$$\text{Filter size} = 3 \times 3$$

$$\text{Stride} = 1$$

$$\text{Padding} = 0$$

- Each filter is applied to all 32 channels.

$$\begin{aligned} 1) \text{ Parameter per filter} &= 3 \times 3 \times 32 \\ &= 288 \text{ weights.} \end{aligned}$$

Each filter has 1 bias term.

$$\begin{aligned} \text{So, total parameters per filter} &= 288 + 1 \\ &= 289 \end{aligned}$$

$$\begin{aligned} 2) \text{ Total parameters of all filters} &= 64 \times 289 \\ &= 18,496 \text{ parameters.} \end{aligned}$$

Q.6 Explain input shape, output shape, filter, padding, stride, tensors.

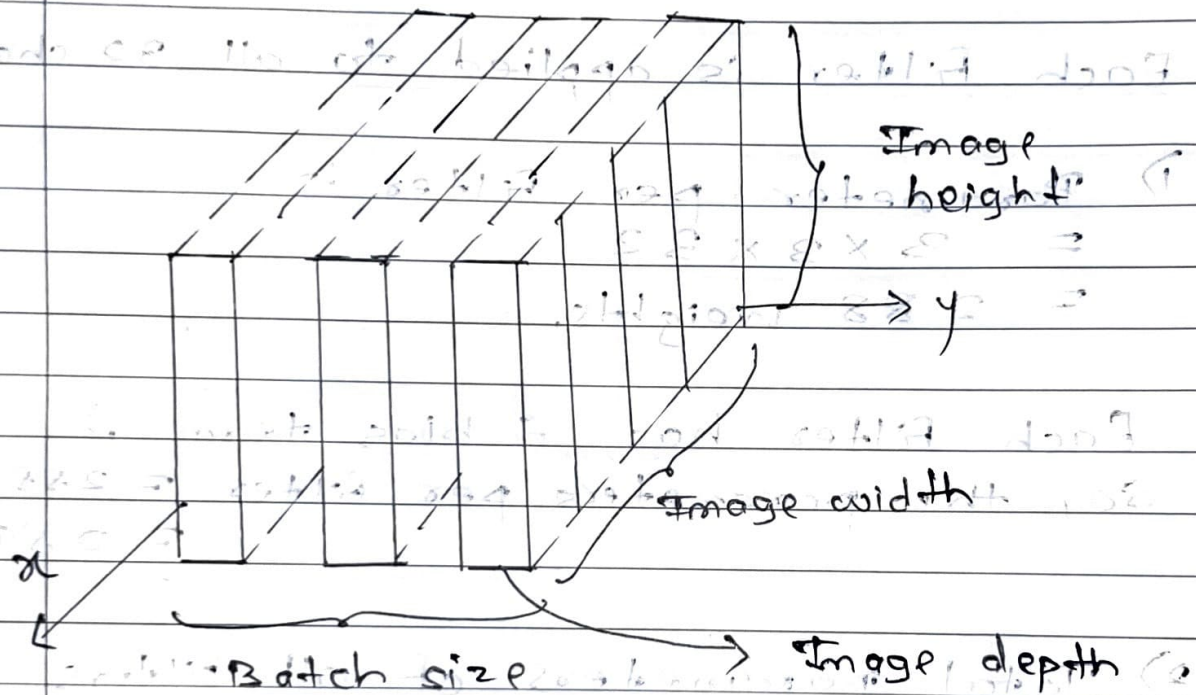
⇒

1) Input Shape

⇒ The input shape refers to the dimension of the data fed into a CNN layer.

— Typically represented as :

{ height × width × channels }



— Batch size is the number of training examples in one forward/backward pass and depth of image is the no. of color channels.

— For e.g., $64 \times 64 \times 3$

2) Output shape

⇒ The output shape is the size of the data it has passed through a convolution or pooling layer.

- Depends on input size, filter size padding and stride.

- Formula :-

$$\text{Output size} = \frac{\text{I/P size} + \text{filter size} + 2P}{S} + 1$$

3) Filter

⇒ A filter is a small matrix that moves over the input to detect patterns.

- Each filter has learnable weights.
- Common filter sizes : 3×3 , 5×5 , etc.
- Filter or kernel matrix moves over the image pixel data.
- It performs a dot product with that particular region of the input data and outputs the matrix of dot product.

4) Tensors

⇒ A tensor is a multi-dimensional array.

- Tensors generalizes scalar, vectors and matrices to higher dimensions.
- Tensors hold inputs, weights and outputs and gradients during training.

Q-7 Explain the process of convolution as a backpropagation.

⇒

- CNN have become the backbone of many modern image processing systems.
- Backpropagation is an algorithm used to calculate the gradients of the loss function of a neural networks with respect to its weight.
- It is essentially a method to update the weight to minimize the loss.
- The process involves three main steps: the forward pass, loss calculations and the backward pass.

1) The Forward Pass:

⇒ During the forward pass, input data is passed through the network to compute the output.

For a CNN, this involves several key operations:

a) Convolutional layers: Each convolutional layer applies numerous filters to the input. For a given layer l with filters denoted by F , input I , bias b , the output O is given by:

$$O = (I * F) + b$$

Here,

$*$ = convolution operation

b) Activation Functions: After convolution, an activation function σ is applied element-wise to introduce non-linearity.

$$O = \sigma(I * F + b)$$

c) Pooling layers: Pooling reduces dimensionality, summarizing the features extracted by the convolutional layers.

2) Loss calculations

⇒ After computing the output, the loss function L is calculated to assess the error in prediction.

Common loss functions include: MSE for regression tasks or cross-entropy for classification.

$$L = - \sum y \cdot \log(\hat{y})$$

Here

y = true label

$\hat{y}$ = predicted label

3) Backward Pass

⇒ The backward pass computes the gradients of the loss function w.r.t each weight in the network by applying the chain rule:

a) Gradient w.r.t output: First, calculate the gradient of the loss function w.r.t the output of final layer: $\frac{\partial L}{\partial O}$

b) Gradient through activation function: Apply the chain rule through the activation function

$$\frac{\partial L}{\partial I} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial I}$$

- For ReLU, $\frac{\partial O}{\partial I}$ is 1 for $I > 0$ and 0 otherwise.

c) Gradient w.r.t Filters in Convolutional layer: Continue applying the chain rule to find the gradients w.r.t filters.

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} * \text{rot180}(I)$$

Here, $\text{rot180}(I)$ rotates the input by 180 degrees.

- Weight Update!

Using the gradients calculated, the weights are updated using optimization algorithm such as SGD.

$$F_{\text{new}} = F_{\text{old}} - \eta \frac{\partial L}{\partial F}$$

- Challenges in Backpropagation:

a) Vanishing Gradients

⇒ In deep networks, backpropagation can suffer from vanishing gradient problem, where gradients become too small to make significant changes in weights.

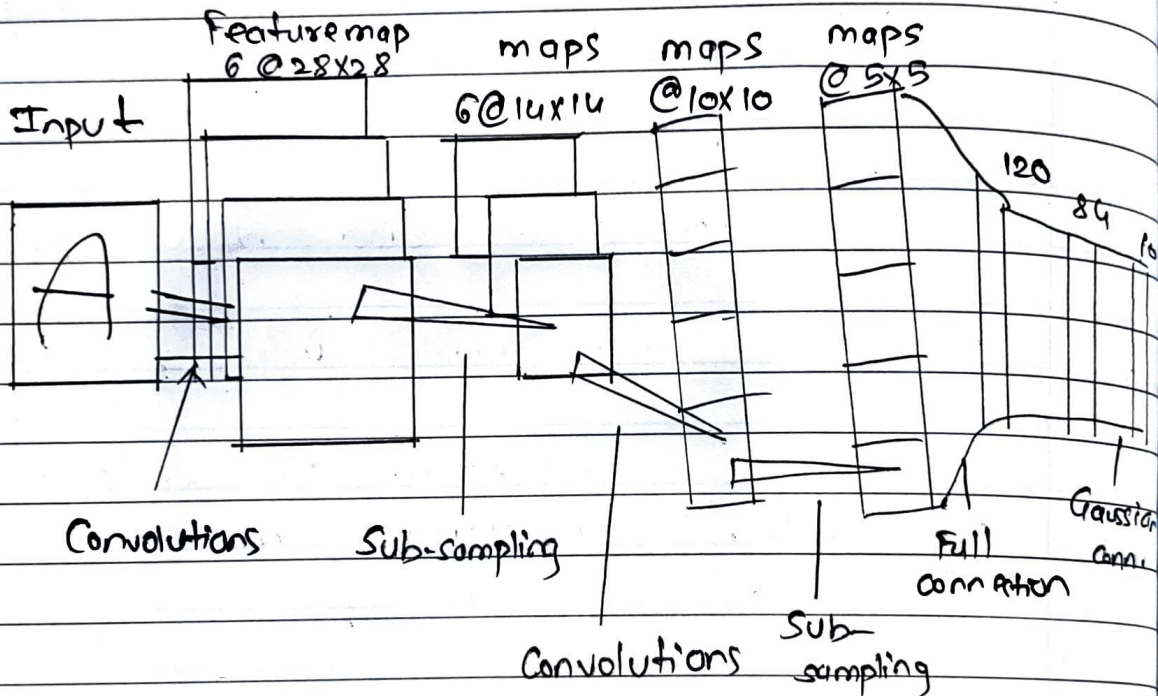
b) Exploding Gradients

⇒ Conversely, gradients become excessively large; this is known as exploding gradients.

Q.8 Describe LeNET architecture.

⇒

- LeNET-5 is a CNN architecture that introduced several key features and innovations that have become standard in modern DL.
- The architecture was designed to identify handwritten digits in the MNIST dataset.
- Architecture :



- The architecture of LeNET-5 contains 7 layers excluding the input layer.

1) Input layer

- ⇒ The input is larger than the largest character in the database, which is at most 20x20 pixels, centered in a 28x28 field.
- The larger input size ensures that distinctive features such as strokes endpoints or corners can appear in the center of respective fields.

2) Layer C1 (Convolution layer)

⇒

- Feature maps: 6 Feature maps
- Connections: Each unit is connected to a 5×5 neighbourhood in the input, producing 28×28 feature maps to prevent boundary effects
- Parameters: 156 trainable parameters and 117,600 connections

3) Layer S2 (Subsampling layer)

⇒

- Feature maps: 6 Feature maps
- Size: 14×14
- Operation: Each unit adds four inputs, multiplies by a trainable coefficient, adds a bias and applies a sigmoid function
- Parameter: 12 trainable parameter and 5,880 connections

4) Layer C3 (Convolution layer)

⇒

- Feature maps: 16 Feature maps
- Connections: Each unit is connected to a 5×5 neighborhoods at identical locations in a subset of S2's Feature maps
- Parameters: 156 trainable parameters and 151,600 connections

5) Layer S4 (Subsampling layer)

⇒

- Feature maps : 16 Feature map.
- Size : 7×7
- Parameters : 32 trainable parameters and 2,744 connections.

6) Layer C5 (Convolution layer)

⇒

- Feature map : 120 Feature map.
- Size : 5×5
- Parameters : 48,000 trainable parameters and 48,000 connections.

7) Layer F6 (Fully connected)

⇒

- Units : 84 units
- Connections : Each unit is fully connected to C5, resulting in 10,164 trainable parameters.
- Activation : Uses a scaled hyperbolic tangent function

$$F(a) = \tanh(a)$$

8) Output layer

⇒ In the output layer, each class is represented by an Euclidean Radial basis function (RBF) unit.

- Formula :

$$y_i = \sum_j x_j \cdot w_{ij}$$

Q.9 Explain and analyze the architecture of AlexNet in CNN.

⇒

- This network is very similar to LeNet-5 but it is deeper with 8 layers: 5 convolutional layer and 3 Fully-connected layer.
- It was trained on two Nvidia GeForce GTX 580 GPUs, therefore, the network was split into two pipelines.
- AlexNet consists of approximately 60M parameters.
- The specific features used are -

a) ReLU Non-linearity

⇒ AlexNet uses ReLU instead of tanh function. ReLU's advantage is in training time.

- CNN using ReLU was able to reach a 25% error on the CIFAR-10 dataset six times faster than CNN using tanh.

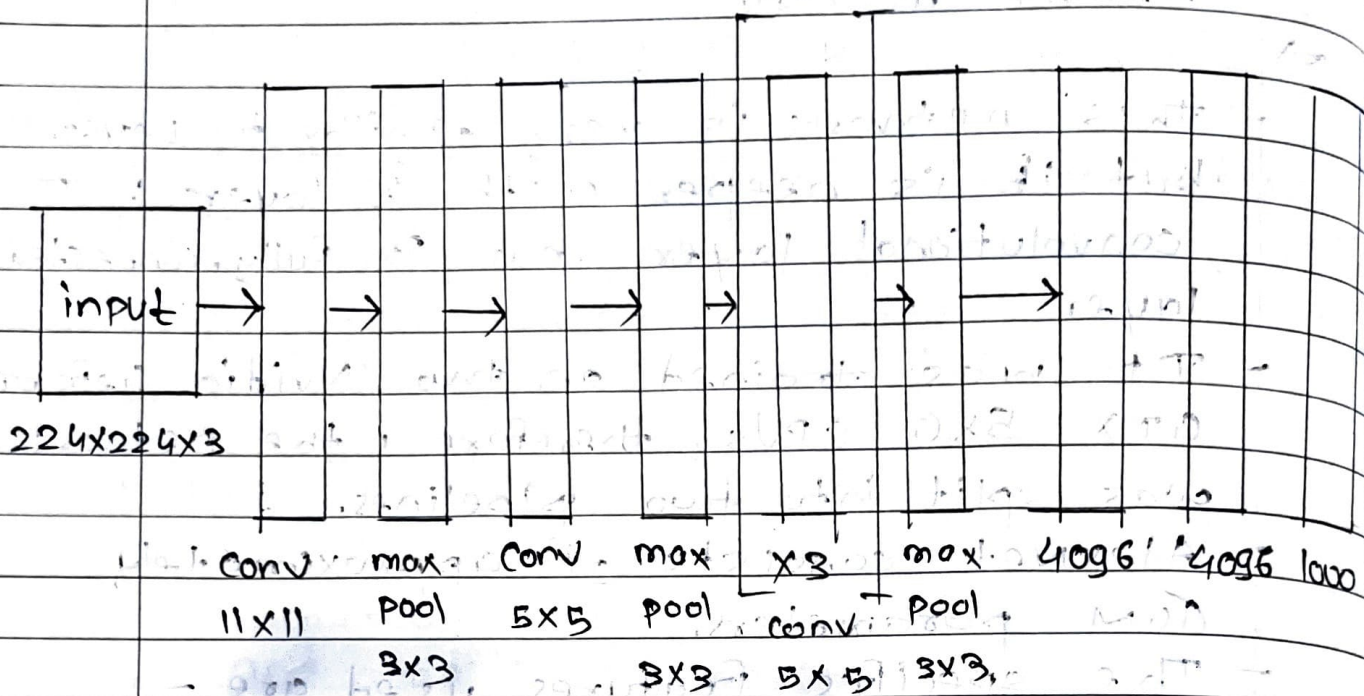
b) Multiple GPUs

⇒ AlexNet allows for multi-GPU training by putting half of the model's neurons on one GPU and the other half on another GPU.

c) Overlapping Pooling

⇒ When the authors introduced overlap, they saw a reduction in error by about 0.5%, and found models with overlapping pooling generally find it harder to overfit.

Architecture



1) Input layer

⇒ The input to the network is an RGB image of size $224 \times 224 \times 3$.

2) Convolutional Layer 1

⇒ The first layer is convolutional layer with 11×11 filters and a stride of 4.

→ This layer helps extract low-level features such as edges and textures.

→ This is followed by a max-pooling layer with 3×3 window that reduces the spatial dimension of the output.

3) Convolution Layer 2

⇒ The second layer is another convolutional layer with 5×5 filters that captures more complex features.

- It is again followed by a 3×3 max pooling layer.

4) Convolution Layer 3, 4 and 5:

→ Three consecutive convolution layers with 5×5 filters are used to extract high-level features such as shapes and object parts.

- These layers are followed by another max pooling layer.

5) Fully Connected Layer

→ The output from convolutional layer is then passed to three fully connected layers.

- The first two fully connected layers each contain 4096 neurons and use ReLU activation, with dropout applied to prevent overfitting.

- The final fully connected layer contains 1000 neurons, corresponding to the number of output classes.

- The output feature map is then passed through Max Pooling layer with pool kernel of 3×3 and stride of 2.

3) Layer 2

⇒ In the second layer, 256 filters are applied of size 3×3 with stride of 2.

- Again Max pooling is applied.

4) Layer 3 & Layer 4

⇒ The third and fourth layer are identical with 384 kernels of size 3×3 and padding is kept same and stride is set to 1.

5) Layer 5

⇒ In the fifth layer, 256 filters of size 3×3 are applied with stride 1.

- Again max pooling is applied.

6) Layer 6 & Layer 7

⇒ The sixth and seventh layer are fully connected dense layers with 4096 neurons each.

7) Eighth layer

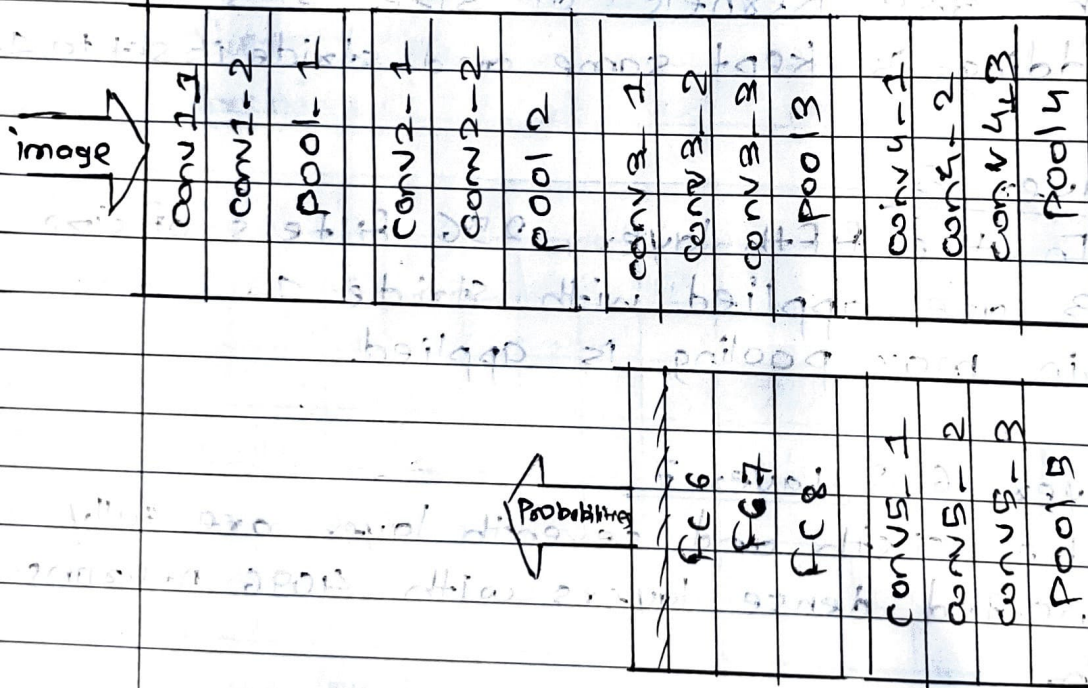
⇒ The last layer is dense layer with 1000 neurons.

2) VGG-Net

⇒ It is a typical deep CNN design with numerous layers and the abbreviation VGG stands for Visual Geometry Group.

- Innovative object identification models are built using the VGG architecture.
- Very tiny convolutional filters are used in construction of the VGG network.
- 13 convolutional layers and 3 fully connected layers make up the VGG-16

Architecture



1) Inputs

⇒ The VGGNet accepts 224×224 images as input.

2) Convolutional layer

⇒ VGG's convolutional layers use the smallest feasible receptive field.

- Additionally, 11 convolution filters are used to transform the input linearly.
- The next component is ReLU unit, a significant advancement from AlexNet that shortens training time.
- The convolution stride is fixed at 1 pixel to keep spatial resolution preserved after convolution.

3) Hidden layers

- ⇒ The VGG network's hidden layers all make use of ReLU.
- Local response normalisation is typically not used with VGG as it increases memory usage and training time.

4) Fully Connected layers

- ⇒ The VGGNet contains three layers with full connectivity.
- The first two levels each have 4096 neurons while the third layer has 1000 channels with one channel for each class.

Q.11 Illustrate GoogLeNet and ResNet deep learning architecture.

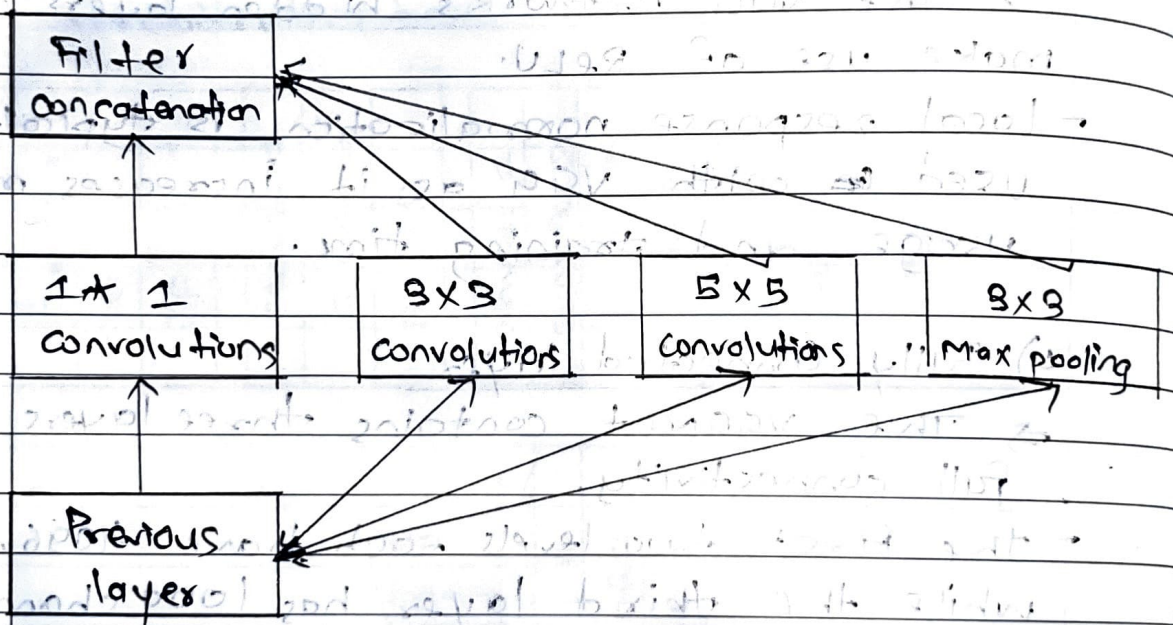
⇒

1) GoogLeNet

⇒ GoogLeNet proposed a novel concept referred to as an inception architecture.

- An inception architecture is a network within a network.

- Architecture :



- The Inception module typically consists of several convolutional layers with different filter sizes.
- These layers are arranged in parallel so that the network can process the input data at multiple resolutions simultaneously.
- The output of the convolution layers is then concatenated and passed through a pooling layer.

- The Inception module allows the network to capture information at different scales.
- The inception module is made up of four paths:

1) 1x1 convolution

- ⇒ This path applies a 1x1 convolution to the input.
- This reduces the no. of channels in the input, which helps to reduce the computational complexity of the network.

2) 3x3 Convolution

- ⇒ This path applies a 3x3 convolution to the input. This is a standard convolution operation that is used to extract features from the input image.

3) 5x5 convolution

- ⇒ This path applies a 5x5 convolution to the input. This path is used to capture larger-scale features from the input image.

4) Max pooling

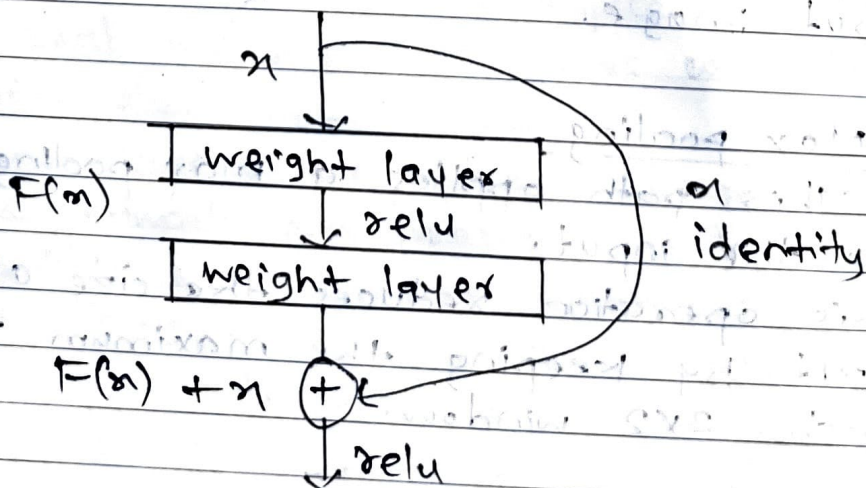
- ⇒ This path applies a max pooling operation to the input.
- This operation reduces the size of the input by keeping the maximum value in each 2x2 window.

2) ResNet

- ⇒ The Residual blocks idea was created by this design to address the issue of the vanishing/exploding gradients.
- We apply a method known as skip connections in this network.
- The skip connection bypasses some levels in between to link-layer activations to subsequent layers.
- This creates a leftover block. These leftover blocks are stacked to create resnets.
- The strategy behind this network is to let the network fit the residual mapping rather than have layers learn the underlying mapping.
- Thus, let the network fit instead of using, say the initial mapping of $H(x)$,

$$F(x) := H(x) - x$$

$$H(x) := F(x) + x$$



- The benefit of including this kind of skip link is that regularization will skip any layer that degrades architecture performance.

Q.14 Difference between fully connected network and CNN:

⇒

Fully Connected Layer

CNN

1) Every neuron is connected to every neuron in the previous layer.

1) Neurons are connected only to a local region of the previous layer.

2) Large number of parameters due to full connectivity.

2) Fewer parameters due to shared weights and local connectivity.

3) No weight sharing; each connection has its own weight.

3) Weights are shared across spatial positions, reducing the no. of parameters.

4) Higher computational cost due to large no. of connections.

4) Lower computational cost per neuron due to local connections.

5) Handles 1D data.

5) Handles high dimensional data 2D images + channels.

FCN

CNN

6) Does not preserve spatial structure of input data.

7) High risk of overfitting.

8) E.g., Multilayer perceptron, Dense Layer in CNNs.

6) Preserves spatial structure of input data.

7) Lower risk of overfitting.

8) E.g., CNNs such as layers in AlexNet, VGGNet

Q.16 Consider the following image of 5×5 and filter size of 3×3 and stride = 1.

Compute following:

i) Output of convolutional operation when padding is applied by following max pooling.

ii) Output of convolutional operation when padding is not applied followed by max pooling.

Filter:	0	-1	0
	-1	3	-1
	0	-1	0

Image:	3	1	3	13	4
	2	12	3	2	12
	3	9	7	12	12
	2	5	11	6	3
	4	12	12	9	3

$\Rightarrow$ Soln:- 5×5 image, 3×3 filter, stride = 1

i)

stride = 1, Input padding = 0, output padding = 1

$$O/p = \frac{5 - 3 + 2(1) + 1}{1} = 2 + 1 = 5 \times 5$$

ii)

0	0	0	0	0	0	0
0	3	1	3	13	4	0
0	2	12	3	2	12	0
0	3	9	7	12	12	0
0	2	5	11	6	3	0
0	4	12	12	9	3	0
0	0	0	0	0	0	0

0	-1	0
-1	3	-1
0	-1	0

6	-15	-8	30	-13
-12	21	-15	-34	18
-4	0	-14	9	9
-6	-9	3	-17	-12
-2	15	4	6	-3

21	30	30
21	21	18
15	15	9

(max pooling stride 1)

21	30
15	9

(Max pooling stride 2)

ii) Input = 5×5 , $F = 3 \times 3$

$$\text{O/p dimension} = \frac{5-3}{1} + 2(0) + 1 = 3 \times 3$$

3	1	3	13	4
2	12	3	2	12
3	9	7	12	12
2	5	11	6	3
4	12	12	9	3

0	-1	0
-1	3	-1
0	-1	0

21	-15	-34
0	-14	9
-19	3	-17

Max pooling = 21

$$\begin{aligned} 1) & (3 \times 0) + (-1) \times 1 + 0 \times 3 + 2 \times (-1) + 3 \times 12 + 3 \times (-1) \\ & + (0 \times 3) + 9 \times (-1) + 0 \times 7 \\ & = -1 - 2 + 36 - 3 - 9 \\ & = 21 \end{aligned}$$

$$2) -3 - 12 + 9 - 2 - 7 = -15$$

$$3) -13 - 3 + 6 - 12 - 12 = -34$$

$$4) -12 - 3 + 27 - 7 - 5 = 0$$

$$5) -3 - 9 + 21 - 12 - 11 = -14$$

$$6) -2 - 7 + 36 - 12 - 6 = 9$$

$$7) -9 - 2 + 15 - 11 - 12 = -19$$

$$8) -7 - 5 + 33 - 6 - 12 = 3$$

$$9) -12 - 11 + 18 - 3 - 9 = -17$$